

אוניברסיטת חיפה
UNIVERSITY OF HAIFA
جامعة حيفا



Software Engineering and Quality Assurance

Department of Information Systems

Panther Group

Member	Student ID	GitHub username
Islam Hassadya	206858334	IslamHassadya11
Fahd Abu Khadra	325558922	Fahidak22
Rami Aboud	214586208	Ramiaboud2005-dotcom
Ghazal Nahhas	212242937	ghazalnah

Repository Link: https://github.com/IslamHassadya11/SE-Ludo-Panther



1. Project Requirements: The system consists of the **following entities**:

1. Board – The track of the chosen level, made up of Squares that are arranged according to the game's difficulty. Its size and special squares vary between levels. **Actions:** Builds the track and places the fixed and the random special squares.

2. **Square** – One position on the Board: regular, protection, start, arrow, home, question or surprise. Its type decides what happens when a Piece lands on it. **Actions**: Holds pieces, checks if a piece may land or pass, and triggers its effect.

3. Piece – One of a player's four pieces; its state (Castle, track, home lane or finished) and position show each player's progress toward the finish. **Actions:** Leaves the Castle, moves along the Board, eats an opponent Pieces, forms Double Blocks with another Piece and reaches the finish.

4. Die – Represents a regular step die, Boost die or UltraBoost die, depending on the game's difficulty. **Actions:** Generates rolls within each die's range and provides the results used to calculate movement.

5. **Player** – A participant with a name, a color and four Pieces. It represents the person in the game and keeps the finishing rank. **Actions:** Rolls the Dice, chooses which Piece to move and answers questions.

6. Castle – The starting area of each Player that holds the pieces before they enter the board and receives them back when eaten. **Actions**: Releases a piece when a 6 is rolled on the regular die and takes back eaten pieces.

7. Question – A multiple-choice question on the course material with four possible answers and a difficulty level. It supports the educational purpose of the game. **Actions:** Checks the selected answer, which determines the Piece's forward or backward movement.

8. Surprise Event – Represents one of six possible effects triggered when a Piece lands on a surprise Square, each with its own probability. **Actions:** Applies its effect to the landing Piece, Player or an opponent's Piece, depending on the event.

9. Admin – The manager of the questions, with a username and a hashed password. **Actions:** Logs in to the question wizard and manages the questions.

10. Question Bank – Stores at least 30 Questions across three difficulty levels in protected storage. It connects the Game, which requests random Questions, with the Admin, who manages them. **Actions:** Retrieves a random Question by difficulty and supports adding, editing and deleting Questions.

11. Game – Runs one match from setup to completion, **connecting its Board, Players and Dice**. It tracks the difficulty level, turns and game progress. **Actions**: Starts the match, enforces game rules, manages turns, determines when the game ends and creates a Game Record.

12. **Game Record** – The saved summary of one finished game: date, level, duration and winners' order. Unlike Game, it stays after the program closes. **Actions:** Records the date, level, duration and winners' order of a finished game in the fixed history schema.

13. Game History – The list of all Game Records, used to keep and show every past game. **Actions**: Adds each new Game Record, saves the history to a file and loads it to list all past games.

Actors: the system has two main actors, a player and an admin. The Player is a person who plays the game.

He starts a new one, rolls the dice, moves pieces, answers questions and views the game history. The admin is

a person who manages the question bank: he logs in to the protected question wizard and views, adds, edits and deletes questions.

Initial Requirements:

#	Requirement	Type	Category
R1	The opening screen allows starting a new game, viewing the history and entering the question wizard.	F	O, I
R2	Before starting a game, the user chooses the number of players, difficulty level, and selects his chosen name and color.	F	O, D
R3	The chosen difficulty level sets the board size, the special squares and the playable dices.	F	O
R4	Protection, question and surprise squares are randomized in each game while the start and arrow squares are always fixed.	F	O
R5	The game screen always displays the board, game time, current player and dice.	F	I
R6	A piece enters the board on a step-die 6; with no active pieces, rolling 1–5 ends the turn.	F	O
R7	Rolling 6 triggers extra roll while a third consecutive 6 is cancelled and ends the turn.	F	O
R8	In the easy level, the steps are the result of the step die. In the medium and hard levels, this result is multiplied by N-1 where N is the Boost/UltraBoost roll respectively.	F	O
R9	Landing on a lone opponent piece on an unprotected square sends it back to its Castle.	F	O
R10	Landing two pieces that belong to the same player on one square form a Double Block that opponents cannot land on or pass through.	F	O
R11	Finishing requires an exact roll. If it is too high, another eligible piece must move; if none can move, the turn ends.	F	O
R12	Question squares show random level-matched questions.	F	O
R13	Correct answers move pieces forward 1, 2 or 3 steps for easy, medium or hard questions while incorrect answers move them backward equally.	F	O
R14	A surprise square triggers one of six events according to their probabilities.	F	O
R15	The game ends when all players except one have brought their 4 pieces to the finish line.	F	O
R16	For each finished game, the winners' order, game level and duration are saved using the prescribed schema given by the course staff (the customer).	F	D
R17	The history screen lists all past games.	F	O, I
R18	Only authorized admins can view, add, edit and delete questions through the wizard.	F	O
R19	For each question, the text, four answers, the correct answer and the level are saved.	F	D
R20	The bank stores at least 30 questions across three difficulty levels.	F	D
R21	Questions and answers are protected against unauthorized access at the data-storage level.	NF	Q
R22	The game is written in Java and utilizes MVC and design patterns.	NF	DI
R23	The game system prevents illegal moves and ends turns with no legal moves.	F	O
R24	Each development round delivers a runnable file tested on multiple computers.	NF	M
R25	The game code is updated weekly on GitHub with collaboration from all team members.	NF	M
R26	The game interface clearly shows available actions and game events.	NF, F	Q, I

Key: F – Functional, **NF** – Non-Functional, **O** – Operational, **D** – Data, **I** – Interface, **P** – Performance, **Q** – Quality, **H** – Hardware, **M** – Management, **DI** – Design and Implementation.